

KAŞ BÖLGESİ ÜZERİNDEN DEEPPFAKE SINIFLANDIRMA KODU

VGG16 (Sıfırdan Eğitim) + HOG + GIST + RBF SVM

Kodun amacı, çalışma mantığı, kullanılan yöntemler ve üretilen çıktılar

Çalışma bölgesi	Kaş (eyebrow ROI)
Model yaklaşımı	Sıfırdan VGG16 özellikleri + el yapımı özellikler + SVM
Çalışma ortamı	Google Colab / Google Drive / Python
Rapor kapsamı	Kod mimarisi ve işlevsel akış

Kaynak notebook: VGG16 (sıfırdan eğitim) + HOG + GIST + RBF SVM.ipynb
Ağustos 2026

1. Yönetici Özeti

Kodun temel amacı

Bu notebook, kaş bölgesine kırılmış gerçek ve sahte frame görüntülerinden üç farklı özellik grubu çıkararak ikili deepfake sınıflandırması yapar. Derin özellikler, önceden eğitilmiş ağırlık kullanılmadan sıfırdan eğitilen VGG16 ağıyla; el yapımı özellikler ise HOG ve GIST-benzeri Gabor tanımlayıcılarıyla üretilir. Bu üç özellik grubu ölçeklendirilip birleştirilir, PCA ile boyut azaltılır ve son karar RBF çekirdekli SVM tarafından verilir.

Kod yalnızca bir model eğitim hücresi değildir; veri denetimi, kaynak video bazlı sızıntı kontrolü, eğitim istatistiklerinin yalnızca train kümesinden hesaplanması, kalite kapıları, atomik checkpoint kaydı, özellik ön bellekleme, validation tabanlı hiperparametre seçimi, frame ve video seviyesinde değerlendirme ve deney çıktılarının denetlenmesi gibi uçtan uca bir araştırma pipeline'i kurar.

Başlık	Açıklama
Girdi	Kaş ROI görselleri, ROI metadata.csv ve frame seçim metadata dosyası
Derin model	VGG16; weights=None ile rastgele başlatılır ve tüm katmanları eğitilir
El yapımı özellikler	HOG ve GIST-benzeri çok ölçekli Gabor özellikleri
Nihai sınıflandırıcı	RBF SVM
Boyut azaltma	Train üzerinde fit edilen PCA; en fazla 256 bileşen
Değerlendirme	Frame seviyesi ve kaynak video seviyesi
Çıktı yapısı	Run ID altında checkpoint, log, metrik, tahmin, grafik ve model artefaktları

2. Uçtan Uca Çalışma Akışı

1	2	3	4	5	6
Metadata kontrolü	VGG16 sıfırdan eğitim	HOG + GIST + VGG özellik çıkarımı	StandardScaler + PCA	RBF SVM seçimi	Test ve çıktı kaydı

- Google Drive bağlanır; kaş görselleri, ROI metadata ve frame seçim metadata dosyalarının varlığı doğrulanır.
- Metadata dosyaları birleştirilir; etiket, split, dosya hash ve kaynak video bilgileri kontrol edilir.
- Train görüntülerinden RGB ortalama ve standart sapma değerleri hesaplanır; validation ve test bu istatistiklerle normalize edilir.
- VGG16 mimarisi önceden eğitilmiş ağırlık olmadan oluşturulur, kalite testlerinden geçirilir ve validation F1 ölçütüyle eğitilir.
- En iyi checkpoint kullanılarak her görüntüden VGG16, HOG ve GIST özellikleri çıkarılır.
- Özellik blokları train verisiyle standartlaştırılır, birleştirilir, PCA uygulanır ve RBF SVM parametreleri validation kümesinde seçilir.
- Seçilen pipeline test kümesinde bir kez değerlendirilir; frame ve video seviyesi sonuçlar ayrı kaydedilir.

3. Veri Girişi, Metadata ve Kalite Kontrolleri

3.1 Kullanılan veri kaynakları

Başlık	Açıklama
Kaş ROI klasörü	Deneyler/Nazlıcan/Deney 1/Kaş
ROI metadata	Kaş/metadata.csv
Split kaynağı	Deney 1 Frame/secim_metadata.csv
Sonuç klasörü	Deneyler/Nazlıcan/Deney 1/Sonuçlar/<run_id>

Kod, kaş görüntülerini salt okunur kabul eder. Görselleri yeniden adlandırmaz, kırpamaz veya ana veri klasörüne yeni özellik görselleri yazmaz. Eğitim ve analiz çıktıları yalnızca benzersiz Run ID ile oluşturulan sonuç klasörüne kaydedilir.

3.2 Metadata birleştirme ve sızıntı önleme

- ROI metadata içindeki başarılı örnekler, yeniden adlandırılmış frame dosya adı üzerinden seçim metadata dosyasıyla eşleştirilir.
- Orijinal frame yolu kullanılarak kaynak video kimliği yeniden elde edilir.
- Train, validation ve test kümelerinde aynı kaynak videonun bulunup bulunmadığı `isdisjoint` kontrolleriyle doğrulanır.
- Aynı içerik hash değerinin farklı splitlerde bulunması engellenir.
- Eksik görsel, eksik kaynak video, yinelenen sample_id, yinelenen dosya hash değeri veya etiket/split uyumsuzluğu varsa eğitim durdurulur.

Neden kaynak video bazlı kontrol yapılıyor?

Aynı videodan alınan birbirine çok benzer frameelerin hem train hem test kümesine düşmesi, modelin deepfake izlerini öğrenmek yerine videoya veya kişiye özgü ayrıntıları ezberlemesine yol açabilir. Kod bu nedenle frame bazlı değil kaynak video bazlı izolasyon uygular.

4. Görüntü Hazırlama ve DataLoader Yapısı

4.1 Letterbox işlemi

Kaş görüntüleri farklı en-boy oranlarına sahip olabileceği için doğrudan kare boyuta esnetilmez. `Letterbox` sınıfı, görüntünün en-boy oranını koruyarak hedef boyuta yerleştirir ve boş kalan alanları doldurur. VGG16 için varsayılan giriş boyutu 128 x 128 pikseldir.

4.2 Train-only normalizasyon

RGB kanal ortalama ve standart sapma değerleri yalnızca eğitim kümesindeki görüntülerden hesaplanır. Bu istatistikler validation ve test kümelerine uygulanır. Böylece test verisinden eğitim sürecine bilgi sızması önlenir.

4.3 Veri artırma işlemleri

Başlık	Açıklama
Yatay çevirme	Olasılık: 0,50
Döndürme	En fazla ± 5 derece

Başlık	Açıklama
Konum değişimi	Görüntü boyutunun %3'üne kadar
Parlaklık değişimi	0,08 düzeyinde ColorJitter
Kontrast değişimi	0,08 düzeyinde ColorJitter
Uygulama alanı	Yalnızca train kümesi; validation ve test deterministiktir

5. Sıfırdan Eğitilen VGG16 Modeli

5.1 Önceden eğitilmiş ağırlık kullanılmaması

Model `vgg16(weights=None)` ifadesiyle oluşturulur. Bu nedenle ImageNet veya başka bir veri kümesinde öğrenilmiş hiçbir ağırlık kullanılmaz. Bütün VGG16 katmanları rastgele başlatılır ve kaş verisi üzerinde eğitilir.

5.2 Mimari düzenleme

Başlık	Açıklama
Convolutional backbone	Torchvision VGG16 features katmanları
Global pooling	AdaptiveAvgPool2d((1, 1))
Derin özellik boyutu	512
Özel sınıflandırıcı	Linear 512→256, ReLU, Dropout(0,40), Linear 256→2
Sınıf sayısı	2: Real ve Fake
Eğitilebilir parametre	Tüm parametreler; notebook çıktısında 14.846.530

5.3 Eğitim ayarları

Başlık	Açıklama
Optimizasyon	AdamW
Başlangıç öğrenme oranı	1e-4
Weight decay	1e-4
Batch size	32
Maksimum epoch	30
Erken durdurma	Validation F1 iyileşmezse 7 epoch sonra
Scheduler	ReduceLROnPlateau; factor=0,5 ve patience=2
Kayıp fonksiyonu	Train sınıf dağılımından hesaplanan ağırlıklı CrossEntropyLoss
Seçim ölçütü	Validation F1

6. Eğitim Öncesi ve Eğitim Sırasındaki Güvenilirlik Mekanizmaları

Başlık	Açıklama
Smoke test	İki batch üzerinde ileri ve geri yayılım yapılır.
Sayısal kontrol	Loss ve gradient değerlerinde NaN/Inf aranır.
Checkpoint testi	Model, optimizier ve durum bilgileri atomik olarak kaydedilir ve tekrar yüklenir.
Çıktı eşdeğerliği	Checkpoint sonrası yeni modelin aynı logits ve loss değerini üretmesi doğrulanır.
Fresh-load inference	En iyi checkpoint sıfırdan kurulan modele yüklenerek tahmin ürettiği kontrol edilir.
Tekrar üretilebilirlik	Python, NumPy, PyTorch ve CUDA seed değerleri sabitlenir.

Atomik kayıt yaklaşımında dosya önce `.tmp` uzantılı geçici bir konuma yazılır, içeriği doğrulanır ve ardından hedef dosyanın yerine geçirilir. Böylece Drive veya çalışma ortamı kesintisinde yarım yazılmış checkpoint oluşma riski azaltılır.

7. Özellik Çıkarma Yöntemleri

7.1 HOG özellikleri

HOG (Histogram of Oriented Gradients), kaş bölgesindeki yerel kenar ve gradyan yönlerini temsil eder. Kaş çizgisi, kıl yönleri ve sınır yapıları bu özellik grubunda öne çıkar. Görüntü 128 x 64 boyutuna en-boy oranı korunarak yerleştirilir ve gri seviyeye dönüştürülür.

Başlık	Açıklama
Yön sayısı	9
Hücre boyutu	8 x 8 piksel
Blok boyutu	2 x 2 hücre
Blok normalizasyonu	L2-Hys
Çıktı boyutu	3.780 özellik

7.2 GIST-benzeri Gabor özellikleri

Kod, klasik GIST yaklaşımına benzer biçimde çok ölçekli ve çok yönlü Gabor filtre cevaplarını uzamsal bölgeler üzerinde özetler. Bu özellikler kaşın genel doku, yön ve uzamsal yerleşim bilgisini temsil eder. Uygulama, notebook içinde açıkça “GIST-style Gabor descriptor” olarak tanımlanmıştır.

Başlık	Açıklama
Görüntü boyutu	128 x 128
Uzamsal grid	4 x 4
Yön sayısı	8
Ölçek sayısı	4
Sigma / dalga boyu çiftleri	(2,4), (3,6), (4,8), (5,10)

Başlık	Açıklama
Çıktı boyutu	4 ölçek x 8 yön x 16 bölge = 512 özellik

7.3 VGG16 derin özellikleri

Eğitimin ardından validation F1 açısından en iyi VGG16 checkpointi yüklenir. Her kaş görüntüsü convolutional backbone'dan geçirilir ve global average pooling sonrasındaki 512 boyutlu vektör alınır. Bu vektör, kaş verisi üzerinde sıfırdan öğrenilmiş derin temsil olarak SVM pipelineına katılır.

Özellik boyutları

HOG: 3.780 + GIST: 512 + VGG16: 512 = toplam 4.804 ham özellik. Her blok önce L2 normalize edilir; ardından StandardScaler, birleştirme ve PCA adımları uygulanır.

8. Özellik Füzyonu, PCA ve RBF SVM

8.1 Özelliklerin hazırlanması

1. HOG, GIST ve VGG16 vektörleri ayrı ayrı L2 normuna göre normalize edilir.
2. Her özellik bloğu için ayrı StandardScaler yalnızca train kümesinde fit edilir.
3. Ölçeklendirilmiş üç blok aynı satır üzerinde birleştirilir.
4. PCA yalnızca train birleşik özellikleri üzerinde fit edilir ve en fazla 256 bileşene indirir.
5. Aynı scaler ve PCA dönüşümleri validation ve test özelliklerine uygulanır.

8.2 SVM hiperparametre seçimi

Başlık	Açıklama
Kernel	RBF
C adayları	0,1; 1; 10; 100
Gamma adayları	scale; 1e-3; 1e-4
Sınıf ağırlığı	balanced
Seçim kümesi	Validation
Seçim ölçütü	F1
Karar eşiği	Validation precision-recall eğrisinde en yüksek F1 veren eşik

SVM, olasılık çıktısı üretmek yerine `decision\_function` skoru üretir. Kod, standart sıfır eşiğini zorunlu kabul etmez; validation kümesinde F1 değerini en yüksek yapan özel karar eşiğini bulur ve test aşamasında bu sabit eşiği kullanır.

9. Değerlendirme Yaklaşımı

9.1 Frame seviyesi

Her kaş ROI görüntüsü bağımsız bir örnek olarak değerlendirilir. Accuracy, precision, recall, F1, specificity, ROC-AUC ve average precision metrikleri hesaplanır. Confusion matrix, ROC eğrisi, precision-recall eğrisi ve karar skoru dağılımı görselleri oluşturulur.

9.2 Video seviyesi

Aynı kaynak videoya ait frame skorları gruplanır ve ortalama karar skoru hesaplanır. Video etiketi bu ortalama skorun validation aşamasında seçilen eşiğe göre karşılaştırılmasıyla üretilir. Böylece frame düzeyindeki dalgalanmaların video kararı üzerindeki etkisi azaltılmaya çalışılır.

10. Üretilen Dosyalar ve Klasörler

Başlık	Açıklama
config_resolved.yaml	Deneyde kullanılan tüm ayarların çözümlenmiş kopyası
checkpoints/best.ckpt	En yüksek validation F1 değerine sahip VGG16 durumu
checkpoints/last.ckpt	Son tamamlanan epoch; devam eğitimi için
logs/training_history.csv	Epoch bazlı loss ve sınıflandırma metrikleri
artifacts/features_*.npz	Train, validation ve test için HOG, GIST ve VGG özellik ön bellekleri
artifacts/ feature_preprocessing.joblib	Train-fitted scaler ve PCA nesneleri
artifacts/svm_model.joblib	Seçilmiş RBF SVM modeli
predictions/*.csv	Frame ve video seviyesi örnek bazlı tahminler
metrics/*.json / *.csv	Veri muhasebesi, kalite kapıları, SVM seçimi ve test metrikleri
figures/*.png	Dağılım, eğitim eğrileri, confusion matrix, ROC ve PR grafikleri
requirements_lock.txt	Çalışma ortamındaki kesin Python paket sürümleri
environment.json	Python, PyTorch, OpenCV, GPU ve platform bilgileri
output_manifest.csv	Üretilen dosyaların boyut ve SHA-256 kayıtları
run_summary.json	Deneyin tek dosyalık genel özeti

11. Kullanılan Teknolojiler ve Kütüphaneler

Başlık	Açıklama
PyTorch / Torchvision	VGG16 modeli, eğitim döngüsü, DataLoader ve GPU kullanımı
OpenCV	Görüntü okuma, letterbox, gri dönüşüm, Gabor filtreleri
scikit-image	HOG özellik çıkarımı
scikit-learn	StandardScaler, PCA, SVC ve değerlendirme metrikleri
Pandas / NumPy	Metadata işleme, özellik matrisleri ve hesaplamalar
Matplotlib	Eğitim ve test grafiklerinin oluşturulması
Pillow	Görüntü dönüştürme ve çıktı çözünürlüğü kontrolü
Joblib / YAML / JSON	Model-ön işleme artefaktları ve konfigürasyon kayıtları
Google Colab / Drive	Çalıştırma ortamı, veri erişimi ve kalıcı çıktı depolama

12. Kodun Güçlü Yönleri ve Teknik Sınırları

12.1 Güçlü yönler

- Derin ve el yapımı özellikleri aynı pipeline içinde birleştirir.
- Önceden eğitilmiş ağırlık kullanmadan gerçek bir sıfırdan eğitim deneyi gerçekleştirir.
- Kaynak video sızıntısını ve aynı içerik hash tekrarlarını kontrol eder.
- Scaler, PCA ve karar eşiği gibi öğrenilen işlemleri train/validation sınırlarına uygun uygular.
- Checkpoint, log, tahmin ve grafik kayıtlarını sistematik bir Run ID altında toplar.
- Kesinti ve bozuk kayıt riskini azaltan atomik yazma mekanizmaları içerir.
- Frame ve video seviyesinde ayrı değerlendirme yapar.

12.2 Teknik sınırlar

- VGG16 sıfırdan eğitildiği için yeterli veri çeşitliliği olmadığında güçlü özellik öğrenmekte zorlanabilir.
- GIST bölümü standart tek bir hazır GIST kütüphanesi yerine Gabor tabanlı GIST-benzeri bir uygulamadır.
- Notebook yalnızca kaş ROI bilgisine dayanır; yüzün diğer bölgelerindeki deepfake ipuçlarını kullanmaz.
- HOG, GIST ve VGG16 özellikleri tek tek ablation deneyi olarak karşılaştırılmadan yalnızca birleşik pipelinein katkısı kesin olarak ayrıştırılamaz.
- Hiperparametre arama uzayı sınırlıdır ve validation kümesine bağlıdır.

13. Genel Sonuç

Bu kod, kaş bölgesinden deepfake sınıflandırması yapmak için hazırlanmış uçtan uca ve izlenebilir bir deney sistemidir. Görüntüleri doğrudan SVM'e vermek yerine önce sıfırdan eğitilen VGG16 ile derin özellikler öğrenir; aynı görüntülerden HOG ile yerel gradyan, GIST-benzeri Gabor yapısıyla genel doku özellikleri çıkarır. Üç özellik grubu train tabanlı ölçeklendirme ve PCA işlemlerinden sonra RBF SVM'e aktarılır.

Notebookun en önemli özelliği yalnızca model kurması değil; veri sızıntısı kontrolünden checkpoint doğrulamasına, validation tabanlı model seçimine, frame/video seviyesinde değerlendirmeye ve yeniden üretilebilir çıktı paketine kadar deneyin tamamını yönetmesidir. Bu nedenle kod, hem model eğitimi hem de araştırma sonuçlarının izlenebilir biçimde saklanması için kapsamlı bir çalışma altyapısı sunar.

Tek cümlelik özet

Kaş framelerinden HOG, GIST-benzeri Gabor ve sıfırdan eğitilmiş VGG16 özellikleri çıkaran; bu özellikleri PCA sonrasında RBF SVM ile gerçek/sahte olarak sınıflandıran ve tüm süreci kontrollü şekilde kaydeden bir deepfake deney notebookudur.

Bu rapor, kullanıcı tarafından sağlanan notebookun kod hücreleri ve gömülü çalışma çıktıları incelenerek hazırlanmıştır.